

	Pràctica Entregable 01 Classe per encriptar informació	IMO30FP01 REV 00 11/13
---	---	---

CURS:	DAM2	DATA:	Setembre-2026	Qualificac ió	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Encriptació i desencriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				
Alumne	Paula Romero García				
Repositori	https://github.com/leralicee/programacio-de-serveis-i-processos/tree/main/PE01_Encryptaci%C3%B3				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRIPATACIÓ → MISSATGE ENCRIPATAT

MISSATGE ENCRIPATAT + CLAU → Procés de DESENCRIPATACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- H → K
- O → R
- L → O
- A → D

Per tant: HOLA → KROD

Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. Investiga el funcionament

Respon aquestes preguntes:

- **Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau?**

Surt exactament el mateix resultat. El sistema és determinista: no té cap part aleatòria, i cada byte del resultat depèn només del caràcter del missatge, de la seva posició i de la clau.

HOLA + clau 3 (1r cop): nYWHmQ==

HOLA + clau 3 (2n cop): nYWHmQ==

Això és el que demana l'enunciat, però també vol dir que qui vegi dos criptogrames iguals sap que el missatge és el mateix.

- **Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.**

El resultat canvia completament:

HOLA + clau 3: nYWHmQ==

HOLA + clau 7: gYmLhQ==

La clau participa tres vegades en cada caràcter: se suma el caràcter de la clau que toca, se suma la suma de tota la clau, i es fa un XOR amb el següent caràcter de la clau. Per això, canviar-ne un sol caràcter afecta totes les posicions del missatge.

- **Què passa si intentem desencryptar utilitzant una clau incorrecta?**

No es recupera el missatge original. El programa no dona cap error, però retorna caràcters sense sentit:

HOLA xifrat amb 3, desxifrat amb 7: <C@=

HOLA xifrat amb 123, desxifrat amb 124: GUJ@

Les operacions inverses (XOR i resta) es fan amb valors que no són els que es van fer servir en encriptar, i per tant cada byte queda desplaçat a un valor diferent. Amb només una xifra diferent a la clau (123 a 124), el resultat ja és irrecuperable.

Com a mínim haurà de complir:

- El mateix missatge amb la mateixa clau ha de produir el mateix resultat.
- Una clau diferent ha de modificar el resultat.
- Si utilitzem la clau correcta, hem de poder recuperar exactament el missatge original.

Descripta(Encripta("HOLA", "123"), "123") = "HOLA"

2. Dissenya l'algoritme propi d'enciptació i desenciptació

Abans de programar, heu de dissenyar el vostre sistema de xifrat.

Podeu utilitzar, per exemple:

- un sistema de substitució.
- un sistema de desplaçament.
- una combinació de diferents operacions.

La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	VX (Vigenère + XOR)
Tipus de sistema	Combinació
Tipus de clau	Text de qualsevol longitud (mínim 1 caràcter). Es treballa amb els bytes UTF-8 de la clau, així que pot tenir lletres, números, símbols o accents. No pot ser buida.
Com funciona l'enciptació?	a sota
Com funciona la desenciptació? Com sé que el meu sistema funciona?	a sota

Com funciona l'enciptació

Primer es passen el missatge i la clau a bytes (UTF-8) i es calcula la suma de tots els bytes de la clau (mòdul 256). Després, per a cada byte del missatge a la posició i:

1. Capa 1 - desplaçament (tipus Vigenère): al byte se li sumen:
 - el caràcter de la clau que toca, $\text{clau}[i \% \text{longitud}]$, que es va repetint en bucle,
 - la posició i, perquè una mateixa lletra no doni sempre el mateix resultat (així HOLA HOLA no repeteix el patró),
 - la suma de tota la clau, perquè equivocar-se en un sol caràcter de la clau canviï totes les posicions.

Si el resultat passa de 255, es fa mòdul 256 perquè torni a cabre en un byte.

2. Capa 2 - XOR: el resultat es combina amb un XOR amb el següent caràcter de la clau, $\text{clau}[(i + 1) \% \text{longitud}]$. Així cada posició depèn de dos caràcters diferents de la clau.

3. Capa 3 - Base64: els bytes resultants no són caràcters imprimibles, i per això es converteixen a Base64 i es poden retornar com a String.

```
xifrat[i] = ((m[i] + clau[i % n] + i + sumaClau) mod 256) XOR clau[(i + 1) % n]
```

Com funciona la descriptació

Es fa el procés al revés, en ordre invers:

1. Es desfà el Base64 per recuperar els bytes xifrats.
2. Es fa el XOR amb el mateix caràcter de la clau, $\text{clau}[(i + 1) \% n]$. El XOR és la seva pròpia inversa: si apliques dues vegades el mateix valor, tornes al número de partida.
3. Es resten els mateixos valors que es van sumar (caràcter de la clau, posició i suma de la clau). Com que el resultat pot ser negatiu, es torna al rang 0..255 amb $((\text{valor} \% 256) + 256) \% 256$.
4. Els bytes es tornen a convertir a text amb UTF-8.

```
m[i] = ((xifrat[i] XOR clau[(i + 1) % n]) - clau[i % n] - i - sumaClau) mod 256
```

Com sé que el meu sistema funciona

- Cada pas té un invers exacte. La suma mòdul 256 es desfà restant mòdul 256, el XOR es desfà amb un altre XOR amb el mateix valor i el Base64 es pot descodificar sense perdre res. Com que la descriptació aplica aquests inversos en ordre contrari, sempre es recupera el missatge original. Exemple fet a mà (apartat 3): HOLA amb la clau 123 dona PSsmOQ==, i en descriptar torna a donar HOLA. El programa dona el mateix resultat que el càlcul manual.
- Proves automàtiques (apartat 5): el ProgramaPrincipal encripta i descripta missatges curts, amb espais i accents, i llargs, i comprova que el recuperat és igual a l'original. També comprova que una clau diferent canvia el resultat i que una clau incorrecta no recupera el missatge.

3. Demuestra el teu algoritme

Missatge: HOLA Clau: 123 → '1' = 49, '2' = 50, '3' = 51 Suma de la clau: 49 + 50 + 51 = 150

Encriptació

Posició	Lletra	Valor	+ clau [i]	+ i	+ suma	Total	mod 256	XOR clau [i+1]	Resultat
0	H	72	+ 49	+ 0	+ 150	271	15	XOR 50	61
1	O	79	+ 50	+ 1	+ 150	280	24	XOR 51	43
2	L	76	+ 51	+ 2	+ 150	279	23	XOR 49	38
3	A	65	+ 49	+ 3	+ 150	267	11	XOR 50	57

Detall dels XOR (en binari):

15 XOR 50 = 001111 XOR 110010 = 111101 = 61

24 XOR 51 = 011000 XOR 110011 = 101011 = 43

23 XOR 49 = 010111 XOR 110001 = 100110 = 38

11 XOR 50 = 001011 XOR 110010 = 111001 = 57

Encriptació: H → 61 O → 43 L → 38 A → 57 Bytes → Base64: [61, 43, 38, 57] → PSsmOQ==
Resultat: PSsmOQ==

Desencriptació

PSsmOQ== → Base64 → [61, 43, 38, 57]

Posició	Byte	XOR clau [i+1]	- clau [i]	- i	- suma	Total	mod 256	Lletra
0	61	XOR 50=15	- 49	- 0	- 150	- 184	72	H
1	43	XOR 51=24	- 50	- 1	- 150	- 177	79	O
2	38	XOR 49=23	- 51	- 2	- 150	- 180	76	L
3	57	XOR 50=11	- 49	- 3	- 150	- 191	65	A

Els negatius es tornen al rang sumant 256: -184 + 256 = 72, i igual amb els altres.

Resultat: HOLA Es recupera el missatge original.

4. Implementació de la classe “Encripta” en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- `Encripta(Missatge, Clau)`
- `Desencripta(Missatge, Clau)`

Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal → ClasseCriptografica → Encripta / Desencripta

5. Proves del sistema dissenyat d'encryptació

Test	Prova	Resultat
Test 1	Missatge curt: HOLA clau 123	 Correcte Xifrat: PSsmOQ== -> recuperat: HOLA. Coincideix amb l'exemple fet a mà.
Test 2	Missatge amb espais: HOLA MÓN clau 123	 Correcte Xifrat: PSsmOd8qolEu -> recuperat: HOLA MÓN. L'espai i la Ó funcionen perquè es treballa amb bytes UTF-8. La Ó ocupa 2 bytes, i per això el missatge de 8 caràcters són 9 bytes. El xifrat comença igual que el de HOLA (PSsmO...), perquè els primers caràcters i la clau són els mateixos.
Test 3	Missatge més llarg: almenys 30 caràcters	 Correcte OglxHQxz3w8LcXYEdQ8NxH8Oy3tzfc11djJmZnpla3s4eXhvfMNSYVFUI1VraVhbVA== -> es recupera exactament. La clau es repeteix en bucle, així que el missatge pot ser més llarg que la clau.
Test 4	Clau diferent: comprovar que el resultat canvia	 Correcte HOLA + 123 = PSsmOQ==, HOLA + 124 = li0oPg==. Canviant només l'últim caràcter de la clau, canvien tots els bytes, gràcies a la suma de la clau.
Test 5	Encriptar i desencriptar: recuperar exactament el missatge original	 Correcte Missatge de prova 2026! amb la clau provaclau -> +MbH2PDFx+8a2MER3s7B3+oD8+307wY= -> Missatge de prova 2026!. El recuperat és igual a l'original (equals retorna true).
Test 6	Clau incorrecta: explicar què passa	 Correcte HOLA xifrat amb 123 i desxifrat amb 124 dona GUJ@. El programa no dona error, però retorna un text sense sentit. Sense la clau correcta no es pot recuperar el missatge.

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard)**, un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

Objectiu

Crear una classe anomenada **ClasseAES** que permeti:

MISSATGE + CLAU



AES



MISSATGE XIFRAT

i recuperar posteriorment el missatge original:

MISSATGE XIFRAT + CLAU



AES



MISSATGE ORIGINAL

La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau) public static  
String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de Java: **Classes:**

```
import javax.crypto.Cipher;  
  
import javax.crypto.spec.SecretKeySpec;  
  
import java.nio.charset.StandardCharsets;  
  
import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()  
  
Cipher.getInstance()  
  
cipher.init()  
  
cipher.doFinal()  
  
Base64.getEncoder()  
  
Base64.getDecoder()  
  
SecretKeySpec  
  
Cipher.ENCRYPT_MODE  
  
Cipher.DECRYPT_MODE
```

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'enciptació i desenciptació

Trobareu un **annex explicatiu** breu al final del document.

6.2. Implementa la classe ClasseAES

Crea una classe independent anomenada `ClasseAES`, que tingui els mètodes:

```
encripta (missatge, clau)  
desencripta (missatge, clau)
```

El mètode `encripta()` haurà de:

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a `String`.
7. Retornar el missatge xifrat.

El mètode `desencripta()` haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.
5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

6.3. Programa principal

Crea un `ProgramaPrincipal` que utilitzi la classe `ClasseAES`. Utilitza inicialment les dades següents:

Missatge: Aquest és un missatge secret.

Clau: 1234567890123456

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

`Desencripta(Encripta(missatge, clau), clau) = missatge`

```
Missatge original: Aquest és un missatge secret.
```

```
Missatge xifrat: 3UXPE1odbnoSr4WWreU9QCSMsetZu7rBCYtwVk16fJc=
```

```
Missatge recuperat: Aquest és un missatge secret.
```

```
Son iguals? true
```

Es compleix que `Desencripta(Encripta(missatge, clau), clau) = missatge`.

El missatge té 29 caràcters, però la “é” ocupa 2 bytes, així que són 30 bytes. Amb el padding queden 2 blocs de 16 (32 bytes), i en Base64 són 44 caràcters.

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

Amb la clau 1234567890123456, el missatge es recupera exactament igual: Aquest és un missatge secret.

Prova 2. Clau diferent

Intenta desxifrar el missatge utilitzant una clau diferent.

Explica què passa.

En desxifrar amb la clau 6543210987654321, que també té 16 bytes però és diferent:

```
Error: Given final block not properly padded. Such issues can arise if a bad key is used during decryption.
```

Java llança una `BadPaddingException`. Amb una clau diferent, AES desxifra els blocs, però el resultat són bytes aleatoris. En acabar, `doFinal()` comprova el padding de l'últim bloc, i com que no té el format PKCS5 correcte, dona error.

En alguns casos (poc freqüents), els bytes aleatoris poden semblar un padding vàlid per casualitat. Llavors no hi ha error, però el text que surt no té sentit. En cap cas es recupera el missatge original.

A diferència del sistema propi, que retorna un text sense sentit sense avisar, AES normalment detecta que alguna cosa ha anat malament.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua funcionant.

```
Original: Demà quedem a les 5 a la biblioteca
```

```
Xifrat:
```

```
Nyx0JHalbTjhow7XY1BGmIJON/owa+yifLIluqwswsBLOM9naK90m+Bc9DKv5E0Iz
```

```
Recuperat: Demà quedem a les 5 a la biblioteca
```

Funciona amb qualsevol missatge i qualsevol longitud, també amb accents. Aquest missatge ocupa 36 bytes, així que queden 3 blocs (48 bytes) i per això el xifrat és més llarg.

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.

Observa què succeeix i explica el motiu.

Amb la clau 1234 (4 bytes):

```
Error: Invalid key length (4).
```

Java llança una `InvalidKeyException` quan es fa `cipher.init(...)`. AES només accepta claus de 16, 24 o 32 bytes (128, 192 o 256 bits), i una clau de 4 bytes no és vàlida. `SecretKeySpec` no comprova la mida; és `Cipher` qui la valida en inicialitzar-se.

També s'ha de tenir en compte que es compten bytes, no caràcters: una clau de 16 caràcters amb accents pot ocupar més de 16 bytes en UTF-8 i també donaria error.

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. Quina classe de Java s'encarrega de realitzar el xifrat?

`javax.crypto.Cipher`. És la classe que implementa l'algoritme (AES en aquest cas) i fa tant l'encriptació com la desenciptació, segons el mode amb què s'inicialitzi.

2. Per a què serveix `SecretKeySpec`?

Converteix un array de bytes en un objecte clau (`SecretKey`) que el `Cipher` pot fer servir. Se li passen els bytes de la clau i el nom de l'algoritme ("AES"). `Cipher.init()` no accepta un `String` ni un `byte[]` directament; necessita un objecte `Key`.

3. Què fa `Cipher.getInstance()`?

Crea un objecte `Cipher` per a la transformació indicada, en format "algoritme/mode/padding", en el nostre cas "AES/ECB/PKCS5Padding". Busca entre els proveïdors de criptografia de Java una implementació d'aquest algoritme. Si no n'hi ha cap, llança una excepció.

4. Quina diferència hi ha entre `ENCRYPT_MODE` i `DECRYPT_MODE`?

Són constants que es passen a `cipher.init()` per dir què farà el `Cipher`. Amb `ENCRYPT_MODE` transforma text pla en dades xifrades i afegeix el padding. Amb `DECRYPT_MODE` fa l'operació inversa: desxifra les dades i treu el padding. Tots dos fan servir la mateixa clau.

5. Per a què serveix `doFinal()`?

Executa l'operació de xifrat o desxifrat sobre les dades que se li passen i la completa. Processa tots els blocs, afegeix el padding (en encriptar) o el comprova i el treu (en desencryptar), i retorna el resultat en un `byte[]`. És el moment en què es fa realment la feina.

6. Per què hem de convertir el missatge a bytes?

AES no treballa amb text, sinó amb blocs de 16 bytes, i fa operacions matemàtiques sobre bytes. Per això `doFinal()` rep i retorna `byte[]`. Amb `getBytes(StandardCharsets.UTF_8)` indiquem una codificació concreta, i així els accents es converteixen igual a qualsevol ordinador i es poden recuperar després amb `new String(bytes, UTF_8)`.

7. Per què utilitzem Base64?

El resultat del xifrat són bytes amb qualsevol valor de 0 a 255, i molts no són caràcters imprimibles. Si els convertíssim directament a `String`, es perdria o es corrompia informació. Base64 representa qualsevol seqüència de bytes només amb caràcters segurs (A-Z, a-z, 0-9, +, / i =). Així el xifrat es pot mostrar, guardar o enviar com a text, i després recuperar els bytes exactes amb el descodificador.

8. Quina mida té la clau que has utilitzat?

La clau 1234567890123456 té 16 caràcters ASCII, que són 16 bytes = 128 bits. Per tant, és AES-128.

9. Què significa que AES sigui un sistema de xifrat simètric?

Que es fa servir la mateixa clau per encriptar i per descriptar. L'emissor i el receptor han de compartir la clau secreta abans de comunicar-se. En els sistemes asimètrics (com RSA), en canvi, hi ha dues claus diferents: una pública per xifrar i una privada per desxifrar.

10. Per què necessitem la mateixa clau per recuperar el missatge?

Perquè AES fa servir la clau per generar les subclaus de cada ronda del xifrat, i la descriptació aplica aquestes mateixes operacions al revés. Només amb la mateixa clau s'obtenen les mateixes subclaus i es desfà exactament la transformació. Amb una altra clau, el resultat són bytes aleatoris, com s'ha vist a la prova 2.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

- Està provat. AES es va triar el 2001 després d'un concurs públic de 5 anys, i des de llavors criptògrafs de tot el món l'han analitzat sense trobar cap atac pràctic. El nostre sistema només l'hem provat nosaltres.
- El sistema propi té debilitats clares. Per exemple, a VX:
 - La clau es repeteix en bucle, com en el xifrat de Vigenère. Es pot atacar buscant patrons que es repeteixen cada n caràcters (mètode de Kasiski).
 - Si un atacant coneix un missatge i el seu xifrat, pot deduir-ne informació de la clau, perquè les operacions són sumes i XOR senzilles.
 - La suma de la clau només pot tenir 256 valors diferents.
 - Missatges que comencen igual donen xifrats que comencen igual (Test 2: HOLA i HOLA MÓN comencen tots dos per PSsmO).
- Principi de Kerckhoffs. Un sistema ha de ser segur encara que l'atacant sàpiga com funciona; només la clau ha de ser secreta. La seguretat de sistemes com el nostre sovint depèn que ningú en conegui l'algoritme, i això no és seguretat de debò.
- Detecta errors. AES amb padding normalment avisa quan la clau és incorrecta, mentre que VX retorna text sense sentit sense avisar.
- Rendiment i compatibilitat. AES està optimitzat, molts processadors el fan per hardware, i qualsevol llenguatge o aplicació el pot fer servir. Qualsevol programa que conegui AES pot llegir un missatge xifrat amb AES; el nostre sistema només l'entén el nostre programa.

Comparació

	Sistema propi (VX)	AES
Tipus	Simètric, de flux (byte a byte)	Simètric, de bloc (16 bytes)
Clau	Qualsevol longitud (≥ 1)	16, 24 o 32 bytes
Seguretat	Baixa, vulnerable a anàlisi de patrons	Alta, estàndard mundial
Clau incorrecta	Retorna text sense sentit	Normalment llança error de padding
Mida del xifrat	Igual que el missatge (+ Base64)	Múltiple de 16 bytes (+ Base64)
Utilitat	Aprendre com funciona un xifrat	Ús real en aplicacions

Milllores per a un ús real d'AES

- Fer servir el mode CBC o GCM amb un IV aleatori en comptes d'ECB, perquè amb ECB blocs iguals donen el mateix xifrat.
- No fer servir directament la contrasenya com a clau, sinó derivar-la amb una funció com PBKDF2 i un salt, com explica l'annex (l'equivalent a Java de `Rfc2898DeriveBytes` és `PBKDF2WithHmacSHA256`).

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- ClasseCriptografica.java
- ProgramaPrincipal.java

2. Implementació AES

- ClasseAES.java
- ProgramaPrincipalAES.java

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Annex Explicatiu

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el mateix criptograma.
- Clau (Key): És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes. **Cipher mode:**

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic.

Clau de xifrat:

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrase que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrase enlloc de una clau i un vector d'inicialització. Utilitzem Rfc2898DeriveBytes per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per RainBow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

Rúbrica:

Criteri	Què s'avalua	Punt s	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	1,5	
2. Implementació del sistema propi	ClasseCriptografica funciona correctament i està separada del programa principal	1,5	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	1,0	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	1,0	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i desencriptació	2,0	
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	1,0	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	0,75	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	0,75	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	0,5	
TOTAL		10	